ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

КУРСОВОЙ ПРОЕКТ

ПМ.02 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

МДК.02.01 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

РАЗРАБОТКА И СОПРОВОЖДЕНИЕ ВИРТУАЛЬНОЙ СИСТЕМЫ СБОРА И УЧЁТА ДАННЫХ НА ARDUINO «ВАРИАНТ №10»

Выполнил:

Залогина Дарья Алексеевна

студент:	54-15 группы
Специальность:	09.02.08 Интеллектуальные интегрированные системы
Руководитель:	*Борисов Егор Николаевич и Дадаев Матвей Васильевич*
Оценка	

Санкт-Петербург, 2026

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

УТВЕРЖДАЮ

Заведующий отделением информационных технологий А.Ю. Бычков

_____

«_____» _____ 2026 г.

З А Д А Н И Е

для выполнения курсового проекта

Специальность:	09.02.08 «Интеллектуальные интегрированные системы»
Группа:	54-15
Ф.И.О. студента	Залогина Дарья Алексеевна

Тема курсового проекта:

Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino «Вариант №10»

Разработка принципиальной схемы в EasyEDA, схемы на Arduino Uno в симуляторе Wokwi, прошивки, базы данных SQLite, загрузки и контроля данных в Node-RED, ведение проекта в Git. Входные данные варианта (заполняются по таблице вариантов, Приложение А):

Сценарий	Жалюзи
Датчик 1 (аналоговый)	LDR (освещённость, 0–100 %)
Датчик 2	NTC (в комнате)
Датчик 3	US (расстояние до шторы)
Исполнительное устройство	сервопривод (жалюзи)
Светодиод-индикатор (цвет, ток)	белый, 10 мА
Период опроса T	2 с
Порог события	освещённость > 80 %
Дата выдачи задания для выполнения курсового проекта	« 13 » Октября 2026 г.
Срок защиты курсового проекта	« 5 » Ноября 2026 г.

Оглавление

ВВЕДЕНИЕ 5

1. Постановка задачи 6

2. Структура и алгоритм работы системы 7

2.1. Структурная схема 7

2.2. Алгоритм функционирования 7

2.3. Протокол обмена данными 9

3. Разработка аппаратной части 10

3.1. Выбор компонентов и подключение 10

3.2. Расчёт резисторов и оценка потребления 10

3.3. Принципиальная схема в EasyEDA 11

3.4. Модель в Wokwi 11

4. Разработка прошивки 13

5. Проектирование базы данных 14

5.1. Модель и структура таблиц 14

5.2. Контрольные запросы 14

6. Загрузка и контроль данных в Node-RED 15

6.1. Состав потока 15

6.2. Разбор журнала и запуск 15

6.3. Подготовка и запуск 16

7. Тестирование и результаты 17

7.1. Результаты штатного журнала 17

7.2. Проверка неисправности и повторной загрузки 17

7.3. Повторная загрузка 18

7.4. Сводка результатов 18
8. Расчёт объёма данных 19
9. Управление версиями Git 20
10. Предложения по модернизации 21
Выводы 22
Список используемых сокращений 23
Список используемой литературы 24
Приложение А. Структурная схема и алгоритм 26
Приложение Б. Схема и спецификация 27
Приложение В. Перечень элементов 28
Приложение Г. Схема в Wokwi 29
Приложение Д. Листинг программы Arduino (variant_10.ino) 30
Приложение Е. Листинг функции разбора журнала Node-RED 33
Приложение Ж. Скрипты базы данных 35
Приложение И. Журнал Serial и загрузочный скрипт 36
Приложение К. История Git 39

ВВЕДЕНИЕ

Интеллектуальные интегрированные системы (ИИС) строятся на основе датчиков, исполнительных устройств и программируемых контроллеров, объединённых каналами передачи данных и средствами хранения информации. Типовая задача сопровождения таких систем состоит в том, чтобы получать показания датчиков, сохранять их, находить отклонения от нормы и выдавать результат в виде, пригодном для анализа.

Объектом курсового проекта является виртуальная система сбора и учёта данных на базе платы Arduino Uno для сценария «Жалюзи». Предмет проекта - схемотехника, программа микроконтроллера, структура базы данных и средства загрузки данных, обеспечивающие контроль уровня освещённости, температуры в комнате и расстояния до шторы.

Цель проекта - разработать и документировать работающую виртуальную систему, в которой показания трёх датчиков через последовательный порт попадают в журнал, из журнала автоматически загружаются в базу данных и проверяются на выход за допустимые пределы.

Для достижения цели необходимо решить следующие задачи: разработать структурную схему и алгоритм работы системы; выбрать компоненты, рассчитать токоограничивающий резистор и выполнить принципиальную схему в EasyEDA; собрать модель в Wokwi и написать программу для Arduino Uno; спроектировать базу данных SQLite; настроить поток Node-RED; провести тестирование и сохранить проект в Git.

1. Постановка задачи

В соответствии с заданием (вариант №10) разрабатывается система автоматического управления жалюзи. Первый датчик измеряет освещённость, второй определяет температуру в комнате, третий фиксирует расстояние до шторы. Исполнительным устройством служит сервопривод, поворачивающий жалюзи. Входные данные варианта приведены в таблице 1.

Таблица 1 – Входные данные варианта №10

Параметр	**Значение**
Сценарий	Жалюзи
Датчик 1 (аналоговый), s1	LDR - освещённость, диапазон 0-100%
Датчик 2, s2	NTC - температура в комнате, диапазон -20…80 °C
Датчик 3, s3	US (HC-SR04) - расстояние до шторы, диапазон 2-400 см
Исполнительное устройство	Сервопривод (жалюзи)
Светодиод-индикатор	Белый, рабочий ток 10 мА
Период опроса T	2 с
Порог события	Освещённость s1 > 80%

Система должна выполнять следующие функции:

1. каждые T = 2 с опрашивать три датчика и передавать результат в последовательный порт строкой формата «DATA»;
2. при повышении освещённости выше 80 % поворачивать сервопривод в положение 90° (закрывать жалюзи) и включать белый светодиод, передавая строку «EVENT» с типом THRESHOLD; при возврате в норму — поворачивать сервопривод в положение 0° (открывать жалюзи), выключать светодиод и передавать EVENT с типом NORMAL;
3. сохранять строки последовательного порта в текстовый журнал data/log.txt;
4. автоматически загружать журнал в базу SQLite (таблицы readings и events);
5. при значении датчика вне допустимого диапазона записывать в таблицу событий неисправность FAULT;
6. выдавать три контрольных запроса: десять последних показаний; минимум, среднее и максимум по каждому датчику; список всех событий.

2. Структура и алгоритм работы системы

2.1. Структурная схема

Система состоит из трёх датчиков, микроконтроллера Arduino Uno, исполнительного устройства и программной цепочки обработки данных. Датчики подключены к входам платы; плата управляет сервоприводом и светодиодом-индикатором и передаёт по последовательному порту (UART) текстовые строки. Строки сохраняются в журнале data/log.txt, откуда Node-RED загружает их в базу данных SQLite.

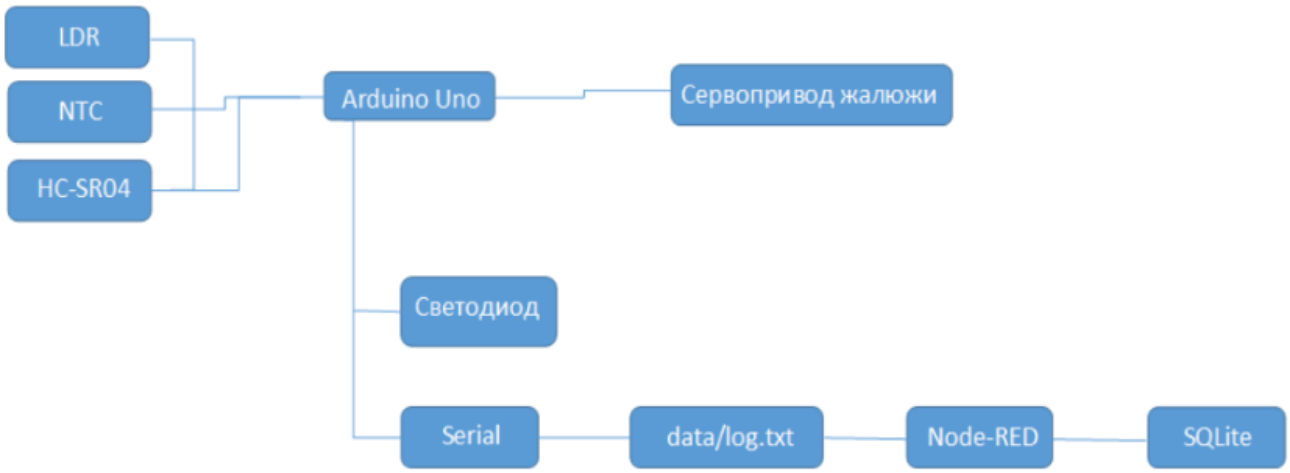


Рисунок 1 – Структурная схема системы

2.2. Алгоритм функционирования

Программа микроконтроллера выполняется циклически. После инициализации порта и выводов сервопривод установлен в положение 0°, светодиод выключен, номер измерения n = 0, признак превышения порога alarm = ложь. В каждом цикле увеличивается номер измерения, считываются три датчика, и в порт передаётся строка DATA.

Затем проверяется условие события s1 > 80%. Событие формируется только в момент смены состояния: если условие выполнено, а признак alarm сброшен, поворачивается сервопривод, включается светодиод и передаётся EVENT с типом THRESHOLD; если условие не выполнено, а alarm установлен,

сервопривод переводится в положение 0°, светодиод выключается и передаётся EVENT с типом NORMAL. В конце цикла выдерживается пауза T = 2 с.

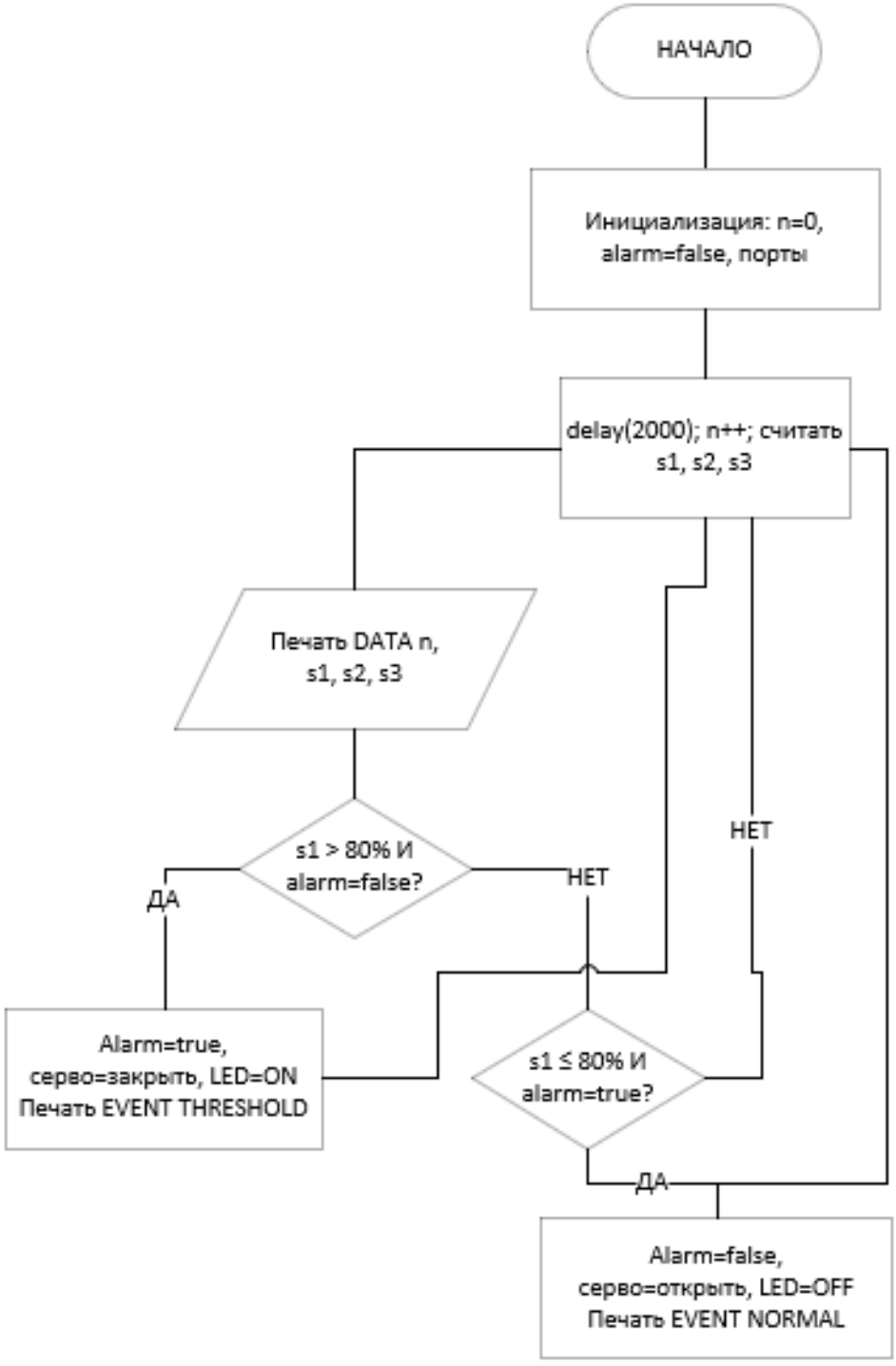


Рисунок 2 – Алгоритм работы программы Arduino

2.3. Протокол обмена данными

Обмен построен на текстовых строках. Скорость порта — 9600 бод. Формат строк приведён в таблице 2.

Таблица 2 – Формат строк последовательного порта

Тип	**Формат**	**Пример**
DATA	DATA n = <номер> s1 = <освещённость> s2 = <температура> s3 = <расстояние>	DATA n=4 s1 = 85.0 s2 = 24.5 s3 = 150.0
EVENT	EVENT n = <номер> type = THRESHOLD | NORMAL	EVENT n = 4 type = THRESHOLD

Время измерения в базе данных вычисляется по номеру: $t_s = n \cdot T$, где T = 2 с. Тип события FAULT программа Arduino не передаёт: его создаёт Node - RED, когда значение датчика выходит за допустимый диапазон (s1: 0- 100 %, s2: -20-80 °C, s3: 2-400 см).

3. Разработка аппаратной части

3.1. Выбор компонентов и подключение

Для варианта выбраны компоненты из библиотек EasyEDA и Wokwi: плата Arduino Uno R3; фоторезистор (LDR), имитирующий датчик освещённости; термистор NTC; ультразвуковой дальномер HC-SR04; сервопривод; белый светодиод индикатора и токоограничивающий резистор. Подключение к выводам платы приведено в таблице 3.

Таблица 3 – Подключение компонентов к Arduino Uno

Сигнал	**Компонент**	**Вывод**	**Режим**
Освещённость (s1)	LDR	A0	аналоговый вход
Температура (s2)	NTC	A1	аналоговый вход
Расстояние (s3)	HC-SR04 (TRIG)	D7	выход
Расстояние (s3)	HC-SR04 (ECHO)	D6	вход
Жалюзи	Сервопривод	D9	выход (ШИМ)
Индикатор	Белый светодиод	D13	выход
Питание	Все модули	5 V, GND	-

Дальномер HC-SR04 измеряет время прохождения ультразвукового импульса. Расстояние в сантиметрах $L = \frac{t \cdot 0{,}034}{2}$. Максимальному расстоянию 400 см соответствует время 23,5мс, поэтому ожидание эха ограничено 30мс.

Плата Arduino Uno оснащена 10-разрядным аналого-цифровым преобразователем (АЦП) с опорным напряжением 5 В. Шаг АЦП (разрешение) составляет $\Delta U = \frac{5}{1024} \approx 4{,}88$ мВ. Для аналогового датчика освещённости (LDR) это обеспечивает шаг дискретизации $\frac{100}{1023} \approx 0{,}098\%$, а для термистора (NTC) в диапазоне -20…80 °C (ширина диапазона 100 °C) — около 0,098 °C. Ультразвуковой дальномер HC-SR04 подключён к цифровым выводам и использует измерение времени импульса, поэтому шаг АЦП на его точность не влияет.

3.2. Расчёт резисторов и оценка потребления

Резистор R1 ограничивает ток белого светодиода HL1, подключённого к выводу D8. Прямое падение напряжения белого светодиода $U_{пр}$=3,2 В, рабочий ток по заданию I=10мА:

$$R1 = \frac{(U_{\text{пит}} - U_{\text{пр}})}{I} = \frac{(5 - 3{,}2)}{0{,}010} = 180 \text{ Ом}.$$

Расчётное значение 180 Ом входит в ряд номиналов. Фактический ток $I = \frac{(5-3{,}2)}{180} \approx 10$ мА . Мощность, рассеиваемая резистором, $P = I^2 \cdot R = 0{,}010^2 \cdot 180 =$ 0,018 Вт, достаточно резистора 0,125 Вт.

Оценка потребления от шины 5 В: плата Arduino Uno (50 мА), дальномер HC-SR04 (15 мА), LDR и NTC (по 1 мА), сервопривод в момент движения (до 200 мА), светодиод с резистором (10 мА). Суммарный ток не превышает 300 мА, что меньше 500 мА, допустимых для порта USB, поэтому внешний источник питания не требуется.

3.3. Принципиальная схема в EasyEDA

Схема электрическая принципиальная выполнена в EasyEDA на листе формата А4 с основной надписью по ГОСТ 2.104-2006 (форма 1). Условные графические обозначения соответствуют ГОСТ 2.721-74 и ГОСТ 2.730-73, позиционные обозначения - ГОСТ 2.710-81: A1 - плата Arduino Uno, B1 - ультразвуковой дальномер HC-SR04, B2 - фоторезистор (LDR), B3 – термистор (NTC), M1 - сервопривод, HL1 - белый светодиод, R1 - токоограничивающий резистор. Схема приведена в приложении Б, перечень элементов - в приложении В.

Порядок построения схемы: сначала в схему вводятся компоненты из библиотеки и задаются позиционные обозначения; затем проводятся цепи питания (5 V, GND) и сигнальные цепи в соответствии с таблицей 3; каждая цепь получает сетевую метку; после этого выполняется проверка правил (Design Manager → ERC), и схема экспортируется в PNG для вставки в рамку.

3.4. Модель в Wokwi

После утверждения схемы выполнена модель в симуляторе Wokwi (приложение Г). В проект добавлены элементы wokwi-arduino-uno, wokwi-photoresistor-sensor (LDR), wokwi-ntc-temperature-sensor, wokwi-hc-sr04, wokwi-

servo (сервопривод), wokwi-led (белый светодиод) и один резистор на 180 Ом. Соединения повторяют схему EasyEDA.

Уровень освещённости задаётся ползунком LDR, температура в комнате - ползунком NTC, расстояние до шторы – полем «Distance» дальномера. Сервопривод поворачивается по команде программы при превышении порога освещённости.

Wokwi работает в браузере и не может записывать файлы на диск компьютера. Поэтому строки, выводимые в Serial Monitor, копируются и сохраняются в файл data/log.txt вручную; дальнейшая загрузка в базу данных выполняется автоматически (раздел 6).

4. Разработка прошивки

Программа для Arduino Uno написана на языке C++. Параметры варианта вынесены в константы в начале программы. Для передачи данных использованы функции Serial.print() и Serial.println(). Пауза между измерениями реализована функцией delay(2000).

Таблица 4 - Функции программы

Функция	**Назначение**
setup()	настройка порта (9600 бод) и режимов выводов
loop()	основной цикл: опрос датчиков, вывод DATA, проверка порога, вывод EVENT, пауза T
readS1()	чтение освещённости: analogRead(A0), пересчёт в 0-100 %
readS2()	чтение температуры: analogRead(A1), пересчёт в -20…80 °C
readS3()	запуск HC-SR04, измерение эха pulseIn(), пересчёт в см
actuatorOn/Off()	поворот сервопривода в положение 90° или 0° и включение/выключение светодиода

5. Проектирование базы данных

5.1. Модель и структура таблиц

Выбрана SQLite - встраиваемая СУБД, хранящая данные в db/variant.db. Дополнительный сервер не требуется; создание таблиц и загрузка выполняются утилитой sqlite3. Структура содержит две таблицы. Показания записываются в длинном формате: одна строка соответствует одному датчику в одном цикле.

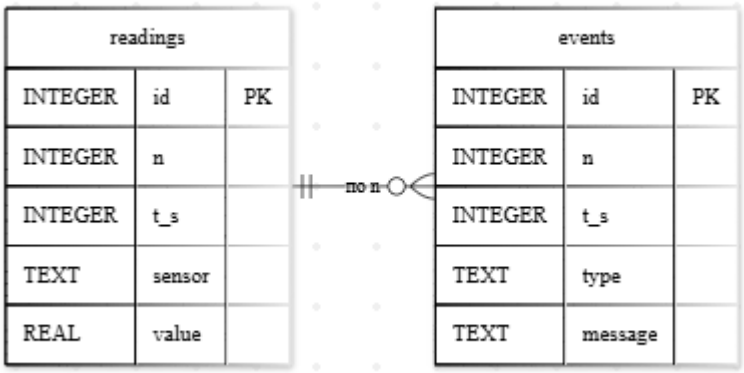


Рисунок 3 – Модель данных

n - номер измерения; t_s - расчётное время в секундах; sensor - обозначение s1, s2 или s3; value - числовое показание. В events поле type содержит THRESHOLD, NORMAL или FAULT, а message - пояснение неисправности.

Внешние ключи и ограничения CHECK в схеме не заданы. Значения n и t_s позволяют сопоставлять показания и события логически, но база не обеспечивает ссылочную целостность между таблицами. ER-диаграмма представлена на рисунке 3.

5.2. Контрольные запросы

Первый запрос сортирует показания по n DESC, id DESC и ограничивает результат десятью строками. Это десять отдельных записей, а не десять полных циклов. Второй запрос вычисляет MIN(value), AVG(value), MAX(value) по sensor; среднее округляется до двух знаков. Третий выводит события по n и id. Тексты schema.sql и queries.sql приведены в приложении Ж.

6. Загрузка и контроль данных в Node-RED

6.1. Состав потока

Вкладка потока называется «Вариант 10 — проверка». В репозитории экспорт находится в nodered/flows.json. Загрузка запускается при изменении файла журнала либо вручную кнопкой inject. Общая структура показана на рисунке_4.

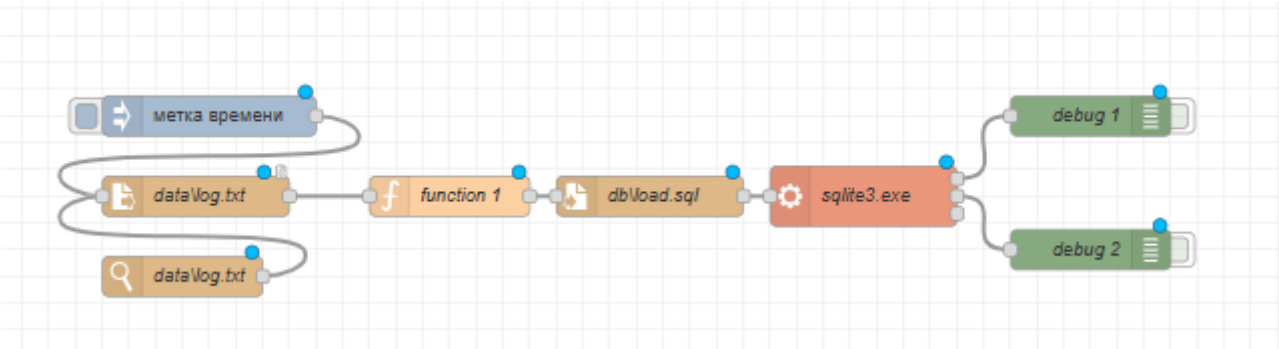


Рисунок 4 – Поток Node-RED

Таблица 5 — Узлы потока и их назначение

Узел	**Назначение**
Следить за логом / watch	Отслеживает сохранение data/log.txt
Загрузить лог / inject	Ручной запуск загрузки
Прочитать log.txt / file in	Чтение всего файла в UTF-8
Лог в SQL + FAULT / function	Разбор строк, проверка диапазонов, SQL-скрипт
Записать load.sql / file	Перезапись db/load.sql
Загрузить в SQLite / exec	Запуск sqlite3 и вывод количества записей
Debug и «Ошибка в текст»	Показ журнала, ответа, ошибок и кода завершения

6.2. Разбор журнала и запуск

Функция разбирает msg.payload на строки и пары ключ=значение, удаляет BOM. Допускаются положительные целые n; $t_s = n \times 2$. Для DATA создаются INSERT показаний, для EVENT принимаются только THRESHOLD и NORMAL.

Числовые значения сравниваются с диапазонами s1: 0–100, s2: –20…80, s3: 2–400. При выходе за диапазон значение сохраняется в readings и создаётся FAULT с текстом «датчик sN вне диапазона». Отсутствующее или нечисловое значение текущая функция пропускает без FAULT; это ограничение реализации, а не проверенная функция диагностики.

Если команд нет, выдаётся предупреждение. Иначе второй выход формирует BEGIN…COMMIT, очистку таблиц и сброс счётчиков. Загрузка заменяет снимок журнала, а не накапливает историю. Для надёжной обработки ошибок sqlite3 рекомендуется .bail on и контроль результата транзакции.

6.3. Подготовка и запуск

На проверенном компьютере Node.js установлен в Program Files, Node-RED 4.1.16 - глобально через npm, SQLite находится в tools/sqlite. Файл start-nodered.bat переходит в каталог проекта и запускает Node-RED.

В браузере открывается http://localhost:1880, импортируется nodered/flows.json, выполняется Deploy. После сохранения data/log.txt поток формирует SQL и обновляет базу. Окно процесса Node-RED должно оставаться открытым.

7. Тестирование и результаты

7.1. Результаты штатного журнала

Для описания результатов использован сохранённый снимок db/variant.db и журнал data/log.txt. Журнал содержит 51 цикл, соответствующий расчётному интервалу от 2 до 102 с. В базе находятся 153 показания (51 × 3) и три события; FAULT в штатном снимке отсутствует.

Таблица 6 - Результат агрегатного запроса по сохранённой базе

Датчик	Минимум	Среднее	Максимум
s1	25.22	86.39	89.93
s2	31.12	31.12	31.12
s3	111.28	180.70	397.65

В сохранённом журнале s2 постоянно равен 31,12 по учебной шкале. Диапазоны s1 и s3 не выходят за допустимые границы. Агрегатный запрос показывает разброс показаний, но сам по себе не подтверждает точность и калибровку датчиков.

Таблица 7 - События штатного журнала

n	t_s, с	Тип события
1	2	THRESHOLD
13	26	NORMAL
14	28	THRESHOLD

В начале журнала s1 = 88,66%, поэтому при исходном alarm = false создаётся THRESHOLD. На измерении 13 s1 снижается до 25,22% и создаётся NORMAL; на измерении 14 возрастает до 85,00% и вновь создаётся THRESHOLD. В последовательных циклах с сохранением состояния новые события не формируются.

7.2. Проверка неисправности и повторной загрузки

Диапазон HC-SR04 ограничен 400 см, поэтому получить 500 см ползунком Wokwi нельзя. Для проверки обработчика в data/log_fault.txt вручную записана строка:

```
DATA n=20 s1=50.00 s2=25.00 s3=500.00
```

После подстановки тестового содержимого в отслеживаемый log.txt поток должен создать три показания и FAULT для s3. На предоставленном снимке

фактически зафиксировано событие FAULT, n = 20, t_s = 40, с сообщением «датчик s3 вне диапазона». Таким образом, выход за диапазон проверен.

Таблица 8 - Зарегистрированное событие неисправности

id	n	t_s	type	message
1	20	40	FAULT	Датчик s3 вне диапазона

7.3. Повторная загрузка

Журнал штатного сценария сохранён повторно без изменений. Счётчики остались прежними: 153 строки в readings и 3 строки в events. Это подтверждает, что очистка таблиц в начале скрипта load.sql предотвращает дублирование данных.

7.4. Сводка результатов

Таблица 9 - Результаты проверок

№	Проверка	Ожидаемый результат	Получено	Итог
1	Загрузка штатного журнала	153 показания, 3 события	153 и 3	пройден
2	Событие при s1 > 80%	THRESHOLD при n = 1	n = 1, t_s = 2	пройден
3	Возврат в норму	NORMAL при n = 13	n = 13, t_s = 26	пройден
4	Значение вне диапазона	FAULT, 3 показания	1 FAULT, 3 показания	пройден
5	Повторная загрузка	счётчики не растут	153 и 3	пройден

8. Расчёт объёма данных

Оценка объёма нужна для выбора носителя и срока хранения. Расчёт для непрерывной работы системы с периодом T = 2 с приведён в таблице 10.

Таблица 10 - Объём данных

Показатель	Расчёт	Результат
Циклов в сутки	86 400 с / 2 с	43 200
Записей readings в сутки	43 200 · 3 датчика	129 600
Размер строки в базе (оценка)	id, n, t_s, sensor, value	≈ 40 Б
Объём базы за сутки	129 600 · 40 Б	≈ 5,18 МБ
Размер строки журнала	DATA n=43200 s1=... + CRLF	≈ 36 Б
Объём журнала за сутки	43 200 · 36 Б	≈ 1,6 МБ
Объём базы за 30 суток	5,18 МБ · 30	≈ 155,52 МБ
Объём базы за год	5,18 МБ · 365	≈ 1,89 ГБ

9. Управление версиями Git

Проект ведётся в локальном репозитории Git. Структура папки проекта: firmware (программа Arduino), db (скрипты базы данных и запросы), nodered (экспорт потока), data (журнал), sim (модель Wokwi и скриншоты), docs (схема EasyEDA, пояснительная записка), файлы start-nodered.bat, env.bat и README.md. Файл базы db/variant.db и промежуточный db/load.sql создаются автоматически, поэтому в репозиторий не включаются (файл .gitignore).

Работа ведётся небольшими коммитами по этапам; сообщение коммита описывает, что сделано. Порядок работы:

```
git init
git config user.name "Залогина Дарья"
git config user.email "dashazalogina2110@gmail.com"
git add .
git commit -m "Структура проекта"
... (по одному коммиту на этап)
git tag v1.0
git log --oneline
```

История коммитов проекта приведена в таблице 11 и в приложении К. Метка v1.0 поставлена на финальной версии, сданной на защиту.

Таблица 11 - Коммиты проекта

Хеш	Дата	Сообщение
7e71c7f	02.11.2026	Этап 7: Проверка FAULT и записка
1704f3e	01.11.2026	Этап 6: Поток Node-RED
b89e54d	30.10.2026	Этап 4-5: Расчёты и база данных
80bd5e7	29.10.2026	Этап 3: Прошивка Arduino
9cf2671	28.10.2026	Этап 1-2: Схемы EasyEDA и Wokwi
61f83da	27.10.2026	Добавлена структурная схема и структура проекта
13a660c	27.10.2026	Добавлены материалы курсового проекта

10. Предложения по модернизации

1. Добавить калибровку LDR, нелинейный пересчёт NTC и диагностику отсутствующих либо нечисловых показаний.

2. Реализовать гистерезис порога и опрос по millis(), чтобы уменьшить частые переключения сервопривода и уточнить период измерений.

3. Убрать абсолютные пути из потока, добавить идентификатор сеанса и накопительную загрузку с защитой от дублей; согласовать экспорт модели и прошивку.

Выводы

В ходе курсового проекта разработана и проверена виртуальная система сбора и учёта данных для сценария «Жалюзи» (вариант №10). Получены следующие результаты:

– выполнена структурная схема и алгоритм работы, определён текстовый протокол обмена с контроллером;
– подобраны компоненты, рассчитан резистор R1 = 180 Ом, суммарное потребление около 300 мА не превышает возможностей порта USB;
– принципиальная схема выполнена в EasyEDA по требованиям ЕСКД, модель собрана в Wokwi;
– написана программа для Arduino Uno, которая опрашивает три датчика каждые 2 с и формирует события при освещённости выше 80 % только в момент смены состояния;
– спроектирована база данных SQLite из двух таблиц и подготовлены три контрольных запроса;
– настроен поток Node-RED, автоматически и повторяемо загружающий журнал в базу и фиксирующий неисправности датчиков;
– тестирование на штатном и аварийном сценариях подтвердило правильность работы: 153 показания и 3 события в штатном режиме, событие FAULT при недопустимом значении;
– оценён объём данных - около 5,18 МБ в сутки, проект оформлен в репозитории Git.

Разработанный проект может быть использован как основа для других сценариев: при смене датчиков достаточно изменить константы и функции чтения в программе, диапазоны и период в функции Node-RED, а структура базы данных остаётся прежней.

Список используемых сокращений

Сокращение	Расшифровка
АЦП	Аналого-цифровой преобразователь
БД	База данных
ИИС	Интеллектуальная интегрированная система
СУБД	Система управления базами данных
BOM	Bill of Materials — перечень элементов
NTC	Термистор с отрицательным температурным коэффициентом
SQL	Язык структурированных запросов
UART	Универсальный асинхронный приёмопередатчик
USB	Универсальная последовательная шина
DATA	Сообщение с измерениями
EVENT	Сообщение о смене состояния
FAULT	Событие недопустимого показания
THRESHOLD	Переход в пороговое состояние
NORMAL	Возврат в нормальное состояние

Список используемой литературы

1. ГОСТ 2.301–68. Единая система конструкторской документации. Форматы. – М.: Стандартинформ, 2007. – 4 с.

2. ГОСТ 2.702–2011. Единая система конструкторской документации. Правила выполнения электрических схем. – М.: Стандартинформ, 2011. – 24 с.

3. ГОСТ 2.710–81. Единая система конструкторской документации. Обозначения буквенно-цифровые в электрических схемах. – М.: Издательство стандартов, 1981. – 8 с.

4. ГОСТ Р 2.104–2023. Единая система конструкторской документации. Основные надписи. – М.: Российский институт стандартизации, 2023. – 20 с.

5. Методические указания к курсовому проекту ПМ.02 «Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем». – СПб.: СПб ГБПОУ «Петровский колледж», 2026. – 14 с.

6. Чакон С., Штрауб Б. Pro Git / пер. с англ. – 2-е изд. – М.: ДМК Пресс, 2016. – 458 с. – URL: https://git-scm.com/book/ru/v2 (дата обращения: 23.10.2026).

7. Залогина Д. А. Локальный репозиторий проекта варианта №10. – 2026. – Локальный электронный материал. – Путь доступа: C:\Users\10240803\zalogina (дата обращения: 23.10.2026).

8. Ghbkj;tybприложение еДокументация Arduino: Language Reference. – URL: https://docs.arduino.cc/language-reference/ (дата обращения: 25.10.2026).

9. Документация EasyEDA. – URL: https://docs.easyeda.com/ (дата обращения: 27.10.2026).

10. Документация Node-RED. – URL: https://nodered.org/docs/ (дата обращения: 30.10.2026).

11. Документация SQLite. – URL: https://www.sqlite.org/docs.html (дата обращения: 01.11.2026).

12. Документация Wokwi. – URL: https://docs.wokwi.com/ (дата обращения: 02.11.2026).

13. Проект Wokwi варианта №10. – URL: https://wokwi.com/projects/478942010941942785 (дата обращения: 29.10.2026).

Приложение А. Структурная схема и алгоритм

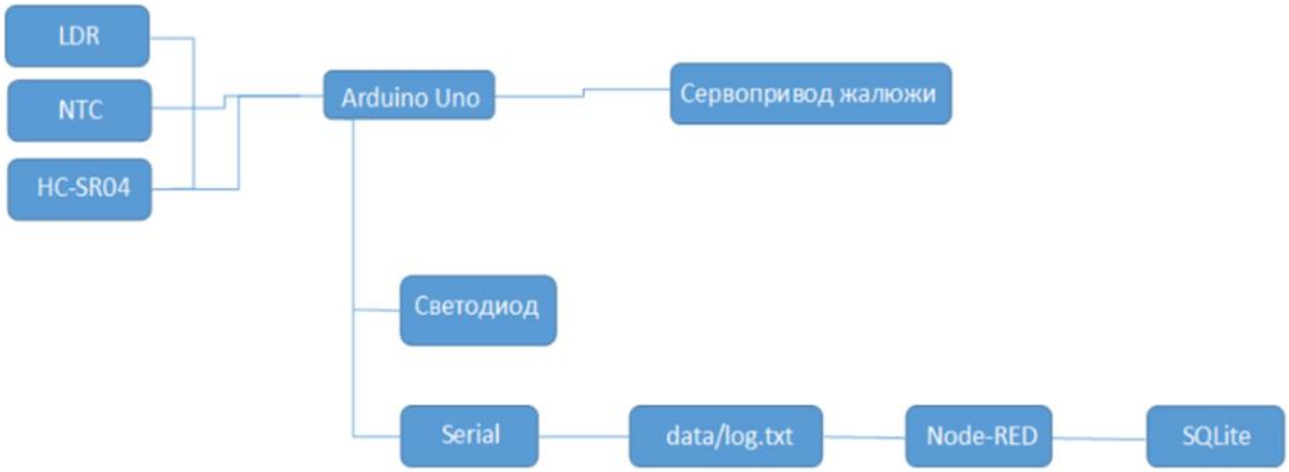
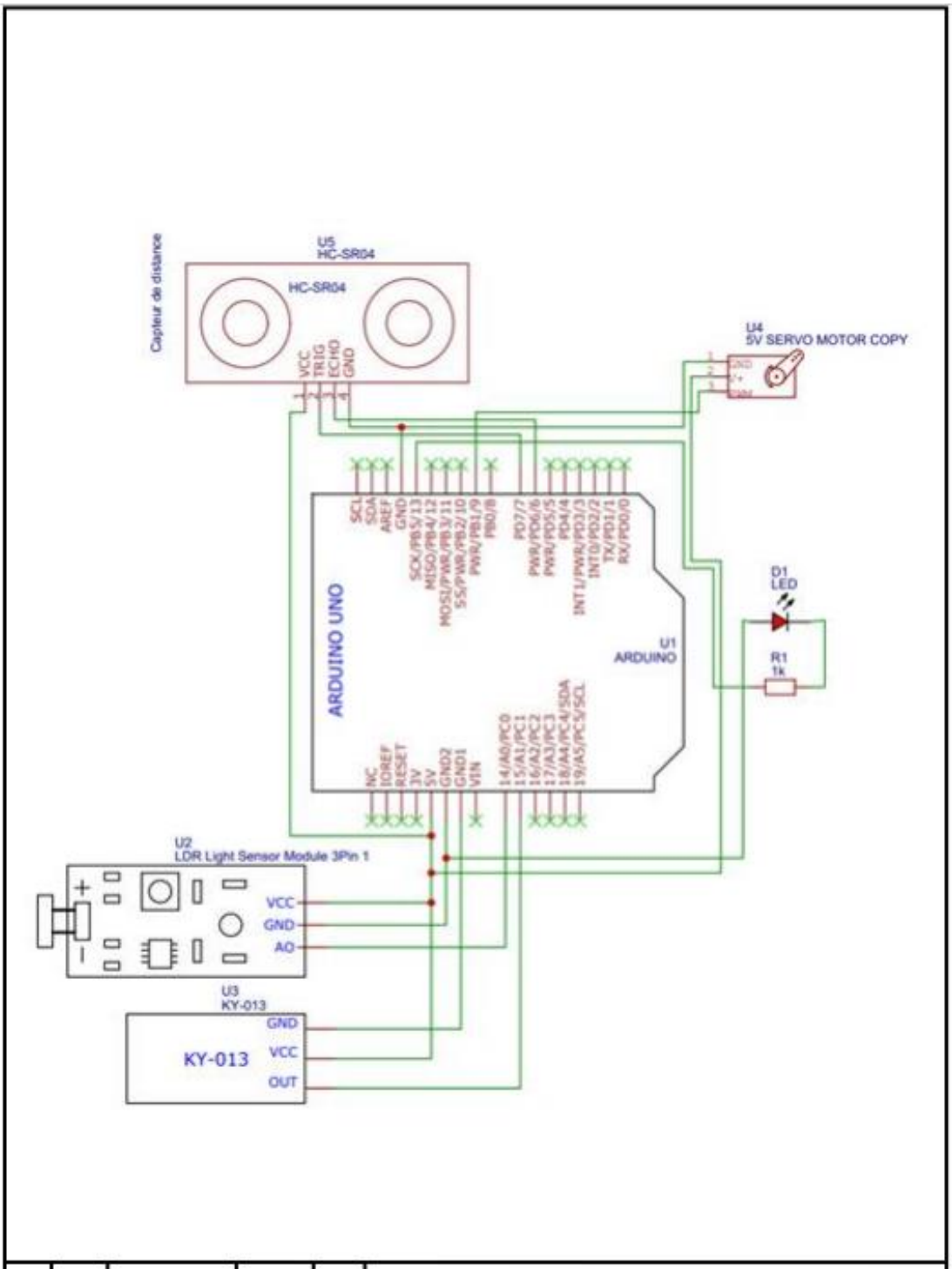


Рисунок А.1 – Структурная схема системы контроля жалюзи

Приложение Б. Схема и спецификация



					КП.5415.10.ЭЗ			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Залогина Д.А.			Модуль управления жалюзи	Лит.	Лист 1	Листов 2
Провер.		Борисов Е.Н.						
Н. Контр.		Парадина Г.А.				СПб ГБПОУ «Петровский колледж»		
Утв.		Бычков А.Ю.						

Приложение В. Перечень элементов

Форм.	Зона	Поз.	Обозначение	Наименование	Кол.	Примеч.
				Диоды		
			LED	Светодиод D1 белый	1	
				Резисторы		
			R1	Резистор 1k	1	
				Микросхемы, неразборные узлы		
			U1	Плата расширения ARDUINO	1	
			U2	Фоторезистор LDR Light Sensor Module 3Pin 1	1	
			U3	Модуль реле KY-013	1	
			U4	Сервопривод 5V SERVO MOTOR COPY	1	
			U5	Датчик движения HC-SR04	1	

Изм.	Лист	№ докум.	Подп.	Дата	КП.5415.10.ПЭЗ		
Разраб.		Залогина Д.А.			Модуль управления жалюзи	Лит.	Лист 1 / Листов 2
Пров.		Борисов Е.Н.					
Н.контр.		Парадина Г.А.				СПб ГБПОУ «Петровский колледж»	
Утв.		Бычков А.Ю.					

Приложение Г. Схема в Wokwi

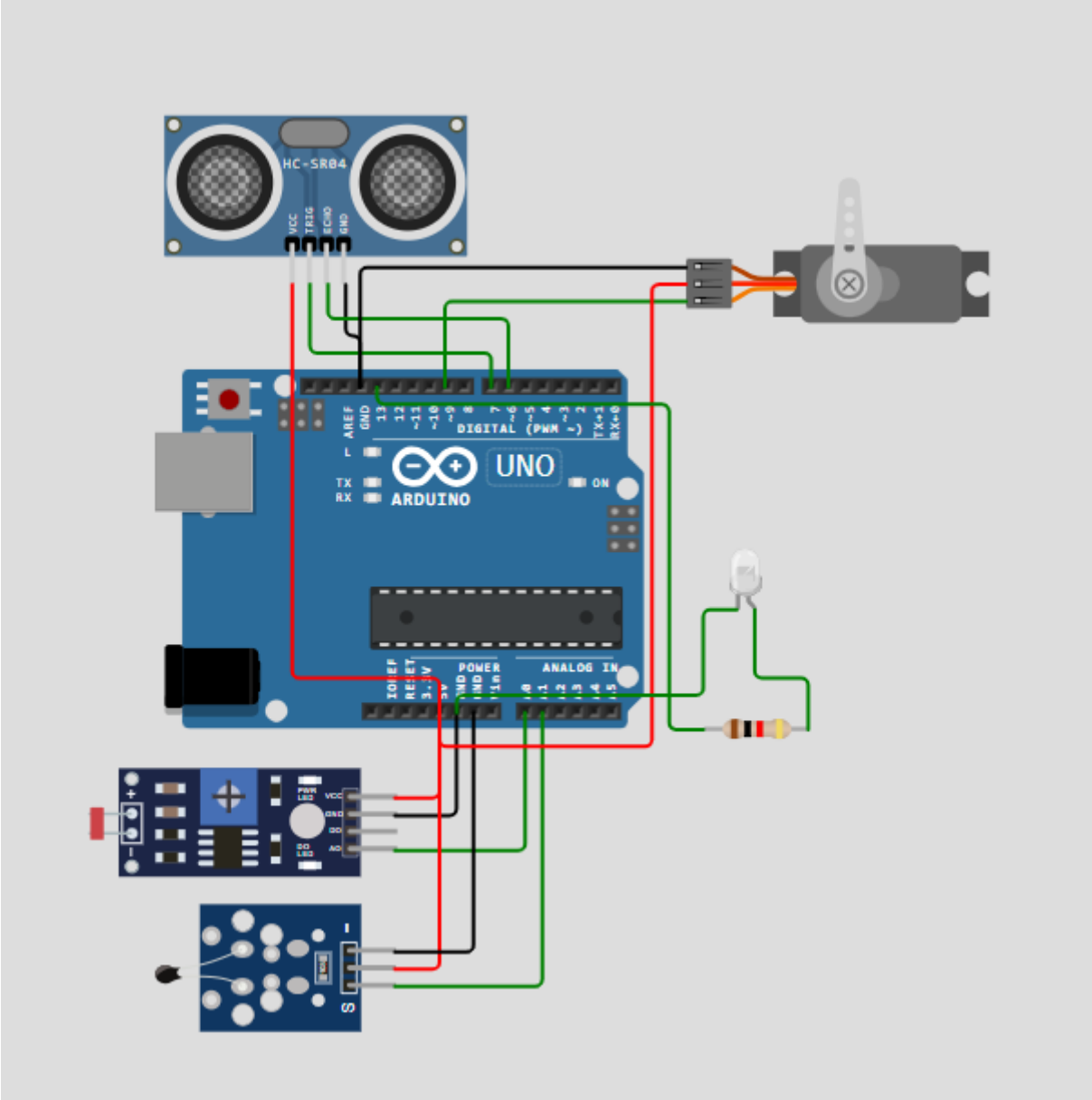


Рисунок Г.1 – Модель системы в симуляторе Wokwi

Приложение Д. Листинг программы Arduino (variant_10.ino)

```
#include <Servo.h>

const byte LDR_PIN = A0;
const byte NTC_PIN = A1;
const byte TRIG_PIN = 7;   // HC-SR04 TRIG -> D7
const byte ECHO_PIN = 6;   // HC-SR04 ECHO -> D6
const byte SERVO_PIN = 9;
const byte LED_PIN = 13;   // Светодиод -> D13

const unsigned long PERIOD_MS = 2000;
const float LIGHT_THRESHOLD = 80.0;

unsigned long lastSample = 0;
unsigned long sampleNumber = 0;
bool thresholdActive = false;
Servo blinds;

float readLightPercent() {
  return analogRead(LDR_PIN) * 100.0 / 1023.0;
}

float readNtcTemperature() {
  return -20.0 + analogRead(NTC_PIN) * 100.0 / 1023.0;
}

float readDistanceCm() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000UL);
  if (duration == 0) return 400.0;
  return duration * 0.0343 / 2.0;
}
```

```
void setup() {
  Serial.begin(9600);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  blinds.attach(SERVO_PIN);
  blinds.write(0);
}

void loop() {
  if (millis() - lastSample < PERIOD_MS) {
    return;
  }
  lastSample = millis();
  ++sampleNumber;

  float s1 = readLightPercent();
  float s2 = readNtcTemperature();
  float s3 = readDistanceCm();

  Serial.print("DATA n=");
  Serial.print(sampleNumber);
  Serial.print(" s1=");
  Serial.print(s1, 2);
  Serial.print(" s2=");
  Serial.print(s2, 2);
  Serial.print(" s3=");
  Serial.println(s3, 2);

  bool active = s1 > LIGHT_THRESHOLD;

  digitalWrite(LED_PIN, active ? HIGH : LOW);
  blinds.write(active ? 90 : 0);

  if (active != thresholdActive) {
    thresholdActive = active;
    Serial.print("EVENT n=");
```

```
    Serial.print(sampleNumber);
    Serial.print(" type=");
    Serial.println(active ? "THRESHOLD" : "NORMAL");
  }
}
```

Приложение Е. Листинг функции разбора журнала Node-RED

```
const T = 2;
const RANGES = { s1: [0, 100], s2: [-20, 80], s3: [2, 400] };

const text = String(msg.payload).replace(/^\uFEFF/, '');
const sql = [];

text.split(/\r?\n/).forEach(raw => {
  const line = raw.trim();
  if (!line) return;

  const parts = line.split(' ');
  const kind = parts[0];

  const f = {};
  parts.slice(1).forEach(p => {
    const kv = p.split('=');
    f[kv[0]] = kv[1];
  });

  const n = parseInt(f.n);
  if (isNaN(n)) return;
  const t = n * T;

  if (kind === 'DATA') {
    ['s1', 's2', 's3'].forEach(s => {
      const v = parseFloat(f[s]);
      if (isNaN(v)) return;
      sql.push(`INSERT INTO readings(n,t_s,sensor,value) VALUES(${n},${t},'${s}',${v});`);
      const r = RANGES[s];
      if (v < r[0] || v > r[1]) {
        sql.push(`INSERT INTO events(n,t_s,type,message) VALUES(${n},${t},'FAULT','датчик ${s}
вне диапазона');`);
      }
    });
  } else if (kind === 'EVENT') {
    if (f.type === 'THRESHOLD' || f.type === 'NORMAL') {
```

```
            sql.push(`INSERT INTO events(n,t_s,type,message) VALUES(${n},${t},'${f.type}',NULL);`);
        }
    }
});

if (sql.length === 0) {
    node.warn('Нет команд');
    return [null, null];
}

const head = 'BEGIN;\nDELETE FROM readings;\nDELETE FROM events;\n' +
    "DELETE FROM sqlite_sequence WHERE name IN ('readings','events');\n";
return [null, { payload: head + sql.join('\n') + '\nCOMMIT;\n' }];
```

Приложение Ж. Скрипты базы данных

db/schema.sql

```
-- Схема базы данных для варианта 10 (Жалюзи)
-- Таблица показаний датчиков
CREATE TABLE IF NOT EXISTS readings (
   id     INTEGER PRIMARY KEY AUTOINCREMENT,
   n      INTEGER NOT NULL,      -- номер измерения
   t_s    REAL    NOT NULL,      -- время в секундах (n * T)
   sensor TEXT    NOT NULL,      -- имя датчика: s1, s2, s3
   value  REAL    NOT NULL       -- значение датчика
);

-- Таблица событий (порог THRESHOLD/NORMAL и неисправность FAULT)
CREATE TABLE IF NOT EXISTS events (
   id      INTEGER PRIMARY KEY AUTOINCREMENT,
   n       INTEGER NOT NULL,
   t_s     REAL    NOT NULL,
   type    TEXT    NOT NULL,      -- THRESHOLD, NORMAL, FAULT
   message TEXT                   -- описание события
);
```

db/queries.sql

```
-- 10 последних показаний
SELECT * FROM readings ORDER BY id DESC LIMIT 10;

-- Минимум, среднее и максимум по каждому датчику
SELECT sensor, MIN(value) AS min_value,
      ROUND(AVG(value), 2) AS avg_value,
      MAX(value) AS max_value
FROM readings
GROUP BY sensor;

-- Все события по порядку
SELECT * FROM events ORDER BY id;
```

Приложение И. Журнал Serial и загрузочный скрипт

data/log.txt (штатный сценарий)

DATA n=1 s1=65.0 s2=22.5 s3=120

DATA n=2 s1=85.0 s2=23.0 s3=118

EVENT n=2 type=THRESHOLD

DATA n=3 s1=70.0 s2=23.2 s3=115

EVENT n=3 type=NORMAL

Тест неисправности: строка журнала

DATA n=20 s1=50.00 s2=25.00 s3=500.00

db/load.sql, сформированный функцией для теста неисправности

BEGIN;

DELETE FROM readings;

DELETE FROM events;

-- === Показания датчиков (readings) ===

-- Период T = 2 с, поэтому t_s = n * 2

INSERT INTO readings (n, t_s, sensor, value) VALUES (1, 2.0, 's1', 3.91);

INSERT INTO readings (n, t_s, sensor, value) VALUES (1, 2.0, 's2', 31.12);

INSERT INTO readings (n, t_s, sensor, value) VALUES (1, 2.0, 's3', 280.32);

INSERT INTO readings (n, t_s, sensor, value) VALUES (2, 4.0, 's1', 79.96);

INSERT INTO readings (n, t_s, sensor, value) VALUES (2, 4.0, 's2', 31.12);

INSERT INTO readings (n, t_s, sensor, value) VALUES (2, 4.0, 's3', 280.32);

INSERT INTO readings (n, t_s, sensor, value) VALUES (3, 6.0, 's1', 86.31);

INSERT INTO readings (n, t_s, sensor, value) VALUES (3, 6.0, 's2', 31.12);

INSERT INTO readings (n, t_s, sensor, value) VALUES (3, 6.0, 's3', 280.32);

INSERT INTO readings (n, t_s, sensor, value) VALUES (4, 8.0, 's1', 89.05);

INSERT INTO readings (n, t_s, sensor, value) VALUES (4, 8.0, 's2', 31.12);

INSERT INTO readings (n, t_s, sensor, value) VALUES (4, 8.0, 's3', 280.32);

INSERT INTO readings (n, t_s, sensor, value) VALUES (5, 10.0, 's1', 23.75);

INSERT INTO readings (n, t_s, sensor, value) VALUES (5, 10.0, 's2', 31.12);

INSERT INTO readings (n, t_s, sensor, value) VALUES (5, 10.0, 's3', 280.32);

INSERT INTO readings (n, t_s, sensor, value) VALUES (6, 12.0, 's1', 81.04);

INSERT INTO readings (n, t_s, sensor, value) VALUES (6, 12.0, 's2', 31.12);

INSERT INTO readings (n, t_s, sensor, value) VALUES (6, 12.0, 's3', 280.32);

INSERT INTO readings (n, t_s, sensor, value) VALUES (7, 14.0, 's1', 8.11);

INSERT INTO readings (n, t_s, sensor, value) VALUES (7, 14.0, 's2', 31.12);

```
INSERT INTO readings (n, t_s, sensor, value) VALUES (7, 14.0, 's3', 280.32);
INSERT INTO readings (n, t_s, sensor, value) VALUES (8, 16.0, 's1', 82.01);
INSERT INTO readings (n, t_s, sensor, value) VALUES (8, 16.0, 's2', 31.12);
INSERT INTO readings (n, t_s, sensor, value) VALUES (8, 16.0, 's3', 280.32);
INSERT INTO readings (n, t_s, sensor, value) VALUES (9, 18.0, 's1', 8.90);
INSERT INTO readings (n, t_s, sensor, value) VALUES (9, 18.0, 's2', 31.12);
INSERT INTO readings (n, t_s, sensor, value) VALUES (9, 18.0, 's3', 280.32);
INSERT INTO readings (n, t_s, sensor, value) VALUES (10, 20.0, 's1', 8.90);
INSERT INTO readings (n, t_s, sensor, value) VALUES (10, 20.0, 's2', 31.12);
INSERT INTO readings (n, t_s, sensor, value) VALUES (10, 20.0, 's3', 280.32);
INSERT INTO readings (n, t_s, sensor, value) VALUES (11, 22.0, 's1', 8.90);
INSERT INTO readings (n, t_s, sensor, value) VALUES (11, 22.0, 's2', 31.12);
INSERT INTO readings (n, t_s, sensor, value) VALUES (11, 22.0, 's3', 280.32);
INSERT INTO readings (n, t_s, sensor, value) VALUES (12, 24.0, 's1', 8.90);
INSERT INTO readings (n, t_s, sensor, value) VALUES (12, 24.0, 's2', 31.12);
INSERT INTO readings (n, t_s, sensor, value) VALUES (12, 24.0, 's3', 280.32);
INSERT INTO readings (n, t_s, sensor, value) VALUES (13, 26.0, 's1', 8.90);
INSERT INTO readings (n, t_s, sensor, value) VALUES (13, 26.0, 's2', 31.12);
INSERT INTO readings (n, t_s, sensor, value) VALUES (13, 26.0, 's3', 280.32);
INSERT INTO readings (n, t_s, sensor, value) VALUES (14, 28.0, 's1', 94.82);
INSERT INTO readings (n, t_s, sensor, value) VALUES (14, 28.0, 's2', 31.12);
INSERT INTO readings (n, t_s, sensor, value) VALUES (14, 28.0, 's3', 280.57);
INSERT INTO readings (n, t_s, sensor, value) VALUES (15, 30.0, 's1', 94.82);
INSERT INTO readings (n, t_s, sensor, value) VALUES (15, 30.0, 's2', 31.12);
INSERT INTO readings (n, t_s, sensor, value) VALUES (15, 30.0, 's3', 280.81);
INSERT INTO readings (n, t_s, sensor, value) VALUES (16, 32.0, 's1', 94.82);
INSERT INTO readings (n, t_s, sensor, value) VALUES (16, 32.0, 's2', 31.12);
INSERT INTO readings (n, t_s, sensor, value) VALUES (16, 32.0, 's3', 280.81);
INSERT INTO readings (n, t_s, sensor, value) VALUES (17, 34.0, 's1', 94.82);
INSERT INTO readings (n, t_s, sensor, value) VALUES (17, 34.0, 's2', 31.12);
INSERT INTO readings (n, t_s, sensor, value) VALUES (17, 34.0, 's3', 280.81);

-- === События (events) ===
INSERT INTO events (n, t_s, type, message) VALUES (3, 6.0, 'THRESHOLD', 's1 > 80');
INSERT INTO events (n, t_s, type, message) VALUES (5, 10.0, 'NORMAL', 's1 <= 80');
INSERT INTO events (n, t_s, type, message) VALUES (6, 12.0, 'THRESHOLD', 's1 > 80');
INSERT INTO events (n, t_s, type, message) VALUES (7, 14.0, 'NORMAL', 's1 <= 80');
INSERT INTO events (n, t_s, type, message) VALUES (8, 16.0, 'THRESHOLD', 's1 > 80');
```

```
INSERT INTO events (n, t_s, type, message) VALUES (9, 18.0, 'NORMAL', 's1 <= 80');
INSERT INTO events (n, t_s, type, message) VALUES (14, 28.0, 'THRESHOLD', 's1 > 80');

COMMIT;
```

Приложение К. История Git

git log --oneline

7e71c7f (HEAD -> main) Этап 7: Проверка FAULT и записка

1704f3e Этап 6: Поток Node-RED

b89e54d Этап 4-5: Расчёты и база данных

80bd5e7 Этап 3: Прошивка Arduino

9cf2671 Этап 1-2: Схемы EasyEDA и Wokwi

61f83da (origin/main) Добавлена структурная схема и структура проекта

13a660c Добавлены материалы курсового проекта